// DEEP DIVE

Quality of Service (QoS)

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

How networks decide which packets get priority when bandwidth runs short — classification, marking, queuing, shaping, and why a video call needs different treatment than a file download.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 — 2	Why QoS exists, and how traffic gets classified & marked
3 — 4	Queuing/scheduling mechanisms, and shaping vs policing
5 — 6	Congestion avoidance (RED/WRED) and the trust-boundary model
7 — 9	QoS for voice/video, a worked configuration example, and reference

How to use this guide: read start to finish for the full picture, or jump to section 3 if you just need the queuing mechanisms compared. Every diagram is numbered and referenced directly in the surrounding text.

// INDEX

Table of Contents

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

// SECTION 1

What Is QoS & Why It Exists

1.1 THE BEST-EFFORT PROBLEM

By default, IP networks are **best-effort** — every packet is treated identically, first-come-first-served, with no guarantee about delivery, delay, or ordering. That's fine as long as there's enough bandwidth for everything trying to cross a link. The moment a link gets congested, best-effort forwarding has no way to distinguish a video call's time-sensitive packet from a background file download's packet — both wait in the same queue, in the order they arrived.

Quality of Service (QoS) is the set of techniques that let a network deliberately treat some traffic better than others — prioritizing, guaranteeing bandwidth to, or protecting latency-sensitive traffic during exactly the moments when a link is under pressure and can't serve everything equally well.

> **QoS doesn't create bandwidth.** It's easy to think of QoS as "making the network faster" — it doesn't. A saturated 100 Mbps link is still only 100 Mbps with QoS enabled. What QoS actually does is control *which traffic gets to use that bandwidth first* when there isn't enough for everyone.

1.2 THE THREE THINGS QOS MANAGES

- **Bandwidth** — how much throughput a traffic class is guaranteed or limited to.
- **Latency & Jitter** — how long a packet waits, and how consistently, before being forwarded. Real-time voice/video is far more sensitive to this than a background download.
- **Packet Loss** — which traffic gets dropped first when a queue fills up completely, since some queue somewhere eventually has to give when there's genuinely more traffic than capacity.

1.3 WHERE QOS MATTERS MOST

QoS is most critical on **constrained or shared links** — a WAN connection between offices, an internet uplink, a congested access-layer switch port — not inside a data center where 10/40/100 Gbps links rarely see real congestion. The golden rule of QoS design: it only ever matters at points of actual or potential congestion. Marking and classifying traffic (Section 2) happens everywhere, but the actual prioritization behavior only kicks in where a link is genuinely constrained.

// SECTION 2

Traffic Classification & Marking

2.1 CLASSIFICATION VS MARKING

Classification is identifying what kind of traffic a packet is — voice, video, bulk file transfer, or best-effort — usually by inspecting the source/destination, port number, or protocol. **Marking** is writing that classification into the packet itself, so every downstream device along the path can make consistent QoS decisions without having to re-classify the traffic all over again from scratch.

2.2 DSCP — THE MODERN MARKING FIELD

The IPv4 header's old 8-bit **ToS (Type of Service)** byte was redefined by **DiffServ** into a 6-bit **DSCP (Differentiated Services Code Point)** field, giving 64 possible marking values. DSCP is the marking most modern networks actually use, since it survives across router hops (unlike Layer 2 markings, which get stripped whenever a frame crosses a Layer 3 boundary).

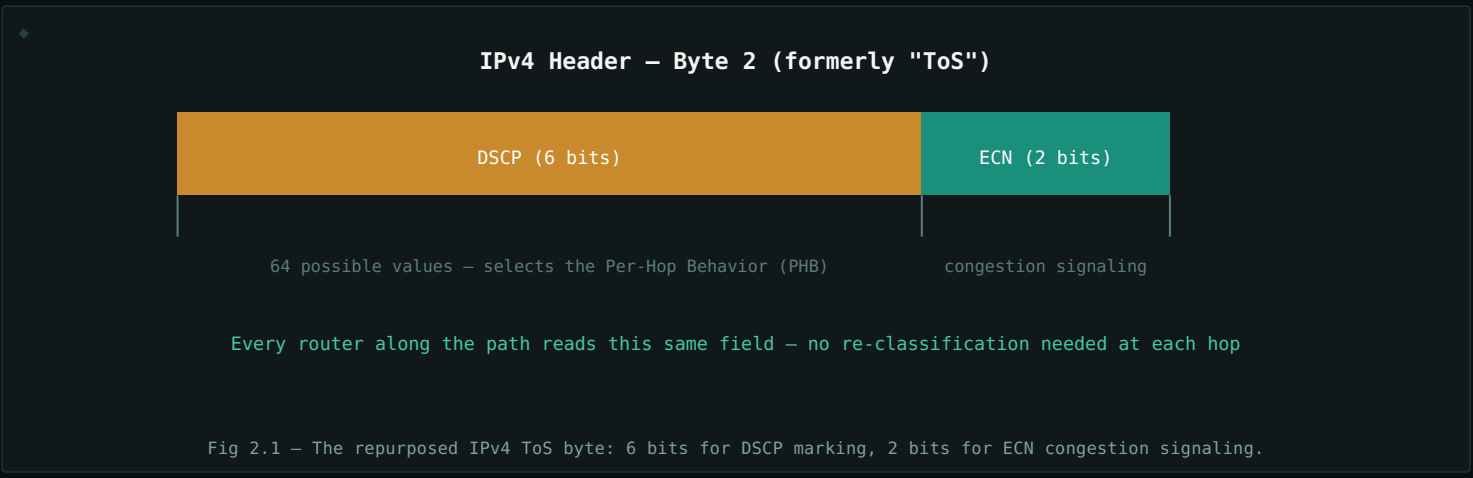

Fig 2.1 — The repurposed IPv4 ToS byte: 6 bits for DSCP marking, 2 bits for ECN congestion signaling.

2.3 COMMON DSCP CLASSES

DSCP CLASS	TYPICAL USE	PRIORITY
EF (Expedited Forwarding)	Voice (RTP media)	Highest — low latency, low jitter, low loss
AF41 / AF42	Interactive video, video conferencing	High — bandwidth-guaranteed, latency-sensitive
AF31 / AF21	Call signaling, business-critical apps	Medium-high
CS1	Bulk data, backups, scavenger traffic	Below best-effort — first to be dropped
Default (0)	Best-effort, unclassified traffic	Standard, no special treatment

2.4 LAYER 2 MARKING: COS

At Layer 2, the 3-bit **CoS (Class of Service)** field lives inside the 802.1Q VLAN tag's PCP bits (see the VLANs & Trunking guide, Section 2, for the full tag layout), giving 8 priority levels. Because it only exists within a VLAN-tagged Ethernet frame, CoS is stripped the moment traffic crosses a router — which is exactly why DSCP, riding inside the IP header itself, is the field that actually matters for end-to-end QoS across a routed network.

// SECTION 3

Queuing & Scheduling Mechanisms

Once traffic is classified and marked (Section 2), a device with multiple traffic classes competing for one outbound link needs an algorithm to decide **which queue gets serviced next**. This is where the actual prioritization happens.

3.1 FIFO — FIRST IN, FIRST OUT

The default, no-QoS behavior: one single queue, packets leave in the exact order they arrived. Simple, but offers zero prioritization — a large file transfer packet and a voice packet wait in the same line.

3.2 PRIORITY QUEUING (PQ)

Multiple queues, strict priority order — the highest-priority queue is always serviced completely before the next queue is touched at all. Excellent for latency-sensitive traffic like voice, but risks **starving** lower-priority queues entirely if high-priority traffic never lets up.

3.3 WEIGHTED FAIR QUEUING (WFQ) & CBWFQ

WFQ automatically shares bandwidth fairly across flows based on weight, without configuration. **CBWFQ (Class-Based WFQ)** extends this with explicit, administrator-defined classes, each guaranteed a minimum bandwidth percentage — no single class can be permanently starved, since every class gets some share even under full congestion.

3.4 LOW LATENCY QUEUING (LLQ)

The design most real networks actually use: a **strict-priority queue reserved exclusively for voice/latency-critical traffic**, sitting on top of CBWFQ for everything else. This gives voice the instant, jitter-free service it needs (like PQ), while still guaranteeing fair bandwidth shares to every other class (like CBWFQ) — combining the strengths of both and avoiding PQ's starvation risk, since the priority queue is typically also policed to a maximum rate so it can't consume the entire link.

Fig 3.1 — Priority Queuing risks starvation; CBWFQ guarantees fair shares; LLQ combines both — strict priority for voice, fair shares for the rest.

MECHANISM	STARVATION RISK?	BEST FOR
FIFO	N/A (no prioritization)	Uncongested links only
Priority Queuing (PQ)	Yes — lower queues can starve	Simple, latency-critical-only scenarios
CBWFQ	No — every class gets a guaranteed share	Fair bandwidth allocation across many classes
LLQ	No — priority queue is policed to a max rate	Real-world voice/video + data mixed networks

// SECTION 4

Traffic Shaping vs Policing

Both shaping and policing enforce a bandwidth limit on a traffic class — but they respond to traffic that exceeds the limit in very different ways.

4.1 POLICING

Policing enforces a hard rate limit by **dropping (or re-marking)** any traffic that exceeds it, immediately, with no buffering. It's simple and doesn't add delay, but it's aggressive — bursts of traffic get discarded outright rather than smoothed out, which can cause retransmissions and choppy throughput for the traffic being policed.

4.2 SHAPING

Shaping enforces the same kind of rate limit, but instead of dropping excess traffic, it **buffers and delays** it, releasing packets at a smoothed, steady rate that fits within the configured limit. This avoids drops (as long as the buffer doesn't overflow) at the cost of adding some latency and jitter to the delayed packets.

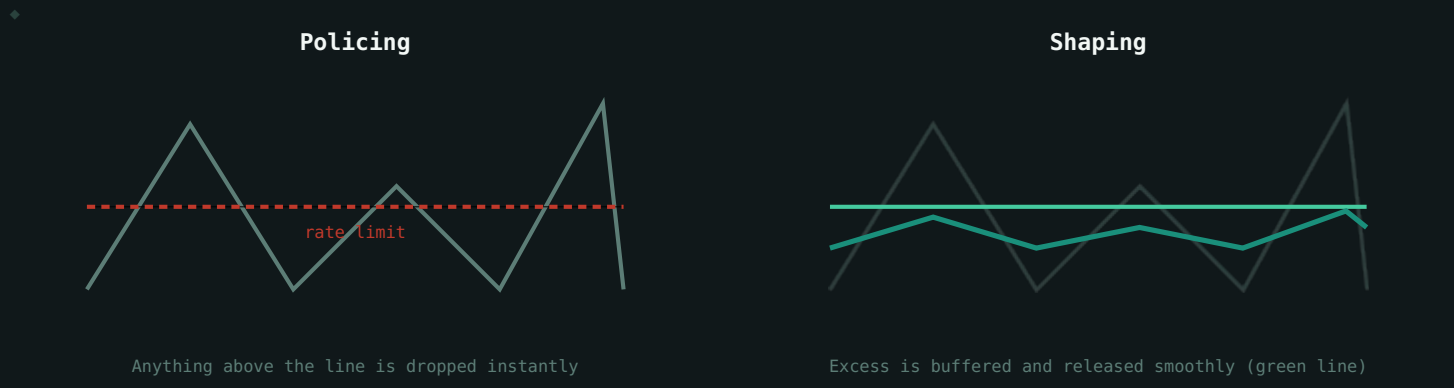

Fig 4.1 — Policing drops traffic above the limit outright; shaping buffers and smooths it into a steady, compliant stream.

4.3 CHOOSING BETWEEN THEM

ASPECT	POLICING	SHAPING
Excess traffic	Dropped or re-marked	Buffered and delayed
Added latency	None	Yes, for buffered packets
Burst handling	Poor — bursts get cut	Good — bursts get smoothed
Typical direction	Inbound, or enforcing an SLA/contract	Outbound, matching a downstream link's known capacity
Common use case	ISP enforcing a customer's purchased bandwidth tier	Matching traffic to a slower WAN link to avoid queue drops downstream

// SECTION 5

Congestion Avoidance (RED & WRED)

5.1 THE PROBLEM WITH LETTING QUEUES FILL COMPLETELY

A naive queue just accepts packets until it's completely full, then drops every new arrival — a behavior called **tail drop**. The problem: when many TCP flows share a link and all hit tail drop at the same moment, they all back off (slow down) simultaneously, then all ramp back up together, then all hit the full queue together again — a synchronized sawtooth pattern called **global synchronization** that wastes available bandwidth on every cycle.

5.2 RED — RANDOM EARLY DETECTION

RED avoids this by starting to **randomly drop a small percentage of packets before the queue is actually full**, once it crosses a minimum threshold. Because the drops are random and start early, only some flows back off at any given moment rather than all of them at once — smoothing out the sawtooth and keeping the queue's average depth (and therefore latency) lower overall.

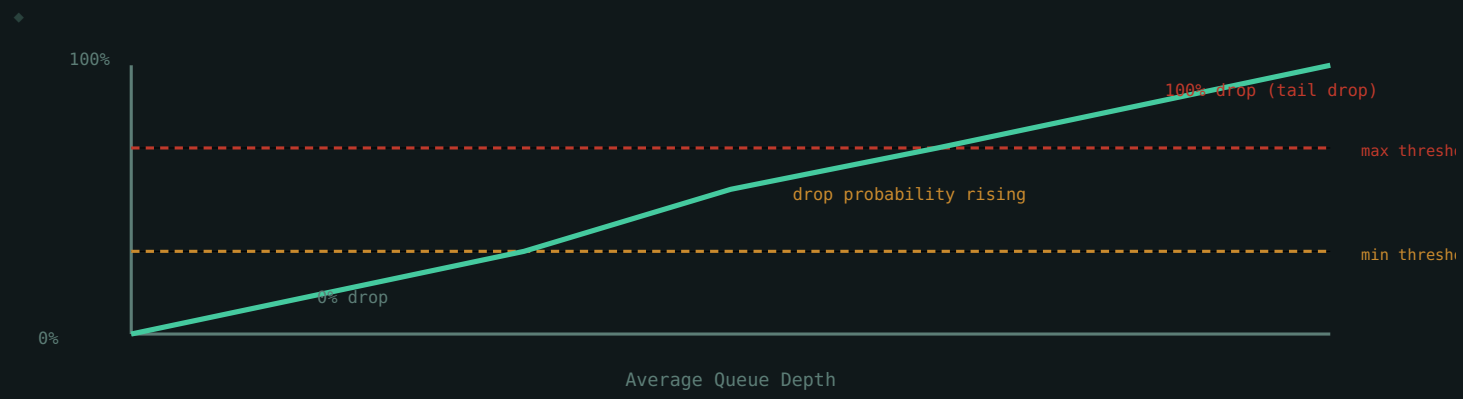

Fig 5.1 — RED's drop probability rises gradually between the min and max thresholds, instead of jumping from 0% to 100% at a full queue.

5.3 WRED — WEIGHTED RED

WRED extends RED by applying **different thresholds and drop probabilities per traffic class** (using the DSCP/CoS markings from Section 2) — lower-priority traffic starts getting dropped earlier and more aggressively, while high-priority traffic gets to use more of the queue before any drops begin at all. This ties congestion avoidance directly into the same classification scheme the rest of the QoS policy already uses.

> **RED/WRED only apply to TCP-friendly traffic.** Dropping packets only helps if the sender actually responds by slowing down — which is exactly what TCP does. UDP-based traffic (like most real-time voice/video, which typically rides in a strict-priority LLQ queue instead) has no such backoff mechanism, so RED/WRED are normally configured only on the queues carrying TCP traffic.

// SECTION 6

Trust Boundaries & the End-to-End Model

6.1 WHY "TRUST" MATTERS

DSCP and CoS markings (Section 2) are just header fields — nothing stops a malicious or misconfigured device from marking all of its own traffic as top priority to selfishly jump the queue. A **trust boundary** is the point in the network where an administrator decides whether to **trust** the markings already on incoming traffic, or to **ignore and re-classify** it from scratch based on more reliable signals (like the specific switch port, VLAN, or application signature it arrived on).

6.2 WHERE TO PLACE THE TRUST BOUNDARY

The general best practice is to classify and mark traffic **as close to the source as possible** — ideally at the access-layer switch port an end device connects to, or even on the end device itself if it's a trusted, managed device like an IP phone. Everything further into the network core then simply trusts and acts on that marking, rather than re-inspecting every packet's actual content at every hop, which would be far more processing-intensive at high-traffic core links.

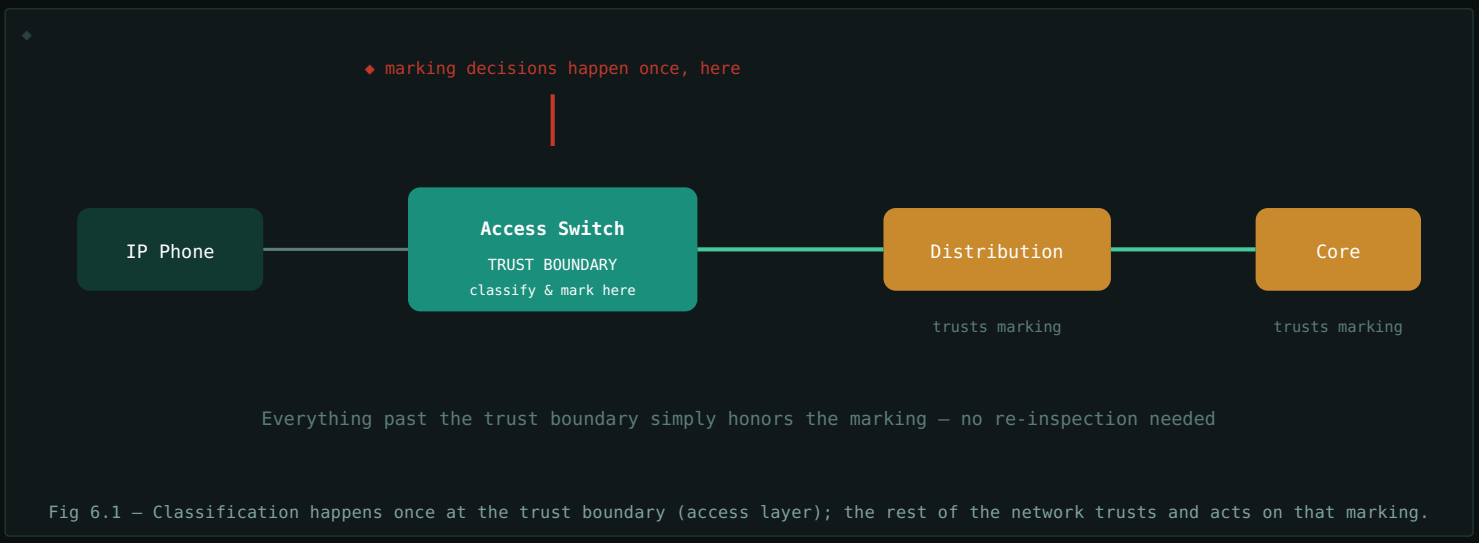

Fig 6.1 — Classification happens once at the trust boundary (access layer); the rest of the network trusts and acts on that marking.

6.3 UNTRUSTED PORTS

Any port a general end user could plug an arbitrary device into — a regular desk PC's data port, a public-facing jack — should generally be configured as **untrusted**, meaning the switch ignores any DSCP/CoS marking already present on incoming traffic and re-marks it according to policy (often just to best-effort or a policy-defined default), preventing users from manually setting their own traffic to a fraudulently high priority.

// SECTION 7

QoS for Voice & Video

7.1 WHY REAL-TIME TRAFFIC IS DIFFERENT

A file download that arrives a little late, or with packets slightly reordered, is barely noticeable — TCP handles reordering and retransmission invisibly. A voice call is the opposite: there's no time to retransmit a lost audio packet before it's needed, and even small, inconsistent delays are audible as choppy or robotic-sounding audio. Real-time traffic cares less about eventually getting everything right, and much more about **consistent, low delay**.

7.2 THE THREE NUMBERS THAT MATTER

METRIC	WHAT IT MEANS	RECOMMENDED TARGET (VOICE)
Latency	One-way delay from sender to receiver	< 150 ms
Jitter	Variation in delay between consecutive packets	< 30 ms
Packet Loss	Percentage of packets that never arrive	< 1%

Why jitter specifically breaks voice: voice codecs expect packets at a steady, predictable interval and use a small "jitter buffer" to smooth out minor timing variation. If jitter exceeds what that buffer can absorb, packets arrive too late to be used and are effectively lost — even though they technically made it across the network. This is why jitter, not just raw latency, is tracked as its own critical metric.

7.3 HOW QOS DELIVERS THESE NUMBERS

- **Classification & marking (Section 2)** — voice (RTP media) is marked EF; call signaling is marked separately (often AF31 or CS3) since it's important but not as delay-sensitive as the media stream itself.
- **LLQ (Section 3)** — voice rides in the strict-priority queue, so it's never stuck waiting behind a large data transfer during congestion.
- **Trust boundary placement (Section 6)** — IP phones are typically trusted devices, marking their own voice traffic correctly right at the source.
- **Avoiding RED/WRED on voice queues (Section 5)** — since UDP-based voice doesn't back off on drops the way TCP does, dropping voice packets early provides no benefit and only hurts call quality.

7.4 VIDEO IS VOICE'S HARDER SIBLING

Interactive video conferencing has similar latency/jitter sensitivity to voice, but with a much larger and more variable bandwidth footprint — a video stream's bitrate can spike significantly during motion-heavy content. This is why video is typically marked into its own class (AF41/AF42) rather than sharing voice's strict-priority queue outright — it gets strong priority and a guaranteed bandwidth allocation, without being able to starve the voice queue during a bandwidth spike.

// SECTION 8

Worked Example: Configuring QoS

A simplified, Cisco MQC-style (Modular QoS CLI) walkthrough implementing the LLQ design from Section 3: voice gets strict priority, video gets a guaranteed share, everything else is best-effort.

8.1 CLASSIFY TRAFFIC INTO CLASSES

```
$ class-map VOICE
$  match dscp ef
$ class-map VIDEO
$  match dscp af41
```

8.2 DEFINE THE POLICY: PRIORITY & BANDWIDTH SHARES

```
$ policy-map WAN-EDGE
$  class VOICE
$   priority percent 15
$  class VIDEO
$   bandwidth percent 25
$  class class-default
$   fair-queue
$   random-detect dscp-based
```

Voice gets a strict-priority queue capped at 15% of the link (policed, per Section 3, so it can't consume the whole link even under a flood of voice traffic). Video gets a guaranteed 25% bandwidth share via CBWFQ. Everything else falls into the default class, sharing what's left fairly, with WRED enabled to avoid TCP global synchronization.

8.3 APPLY THE POLICY TO AN INTERFACE

```
$ interface GigabitEthernet0/0
$  service-policy output WAN-EDGE
```

The policy is applied **outbound** on the WAN-facing interface — the point where this link's limited bandwidth actually becomes a constraint, and where queuing decisions matter (per the Section 1.3 rule: QoS only matters at points of real or potential congestion).

8.4 VERIFYING THE POLICY

```
$ show policy-map interface GigabitEthernet0/0
```

This shows real-time statistics per class — packets matched, dropped, and current queue depth — the first place to check when a QoS policy doesn't seem to be behaving as expected.

> **In practice:** exact syntax varies by vendor, but the underlying MQC-style pattern — define classes, build a policy assigning treatment to each class, apply the policy to an interface in the correct direction — is the near-universal structure QoS configuration follows across platforms.

// SECTION 9

Quick Reference & Glossary

TERM	DEFINITION
QoS	Quality of Service — techniques for prioritizing certain traffic during network congestion.
Best-Effort	Default IP forwarding behavior — all traffic treated identically, no guarantees.
DSCP	Differentiated Services Code Point — a 6-bit IP header field marking a packet's traffic class.
CoS	Class of Service — a 3-bit Layer 2 priority field inside the 802.1Q VLAN tag.
EF	Expedited Forwarding — the DSCP class typically used for voice, the highest common priority.
Classification	Identifying what kind of traffic a packet is.
Marking	Writing a classification decision into the packet header (DSCP/CoS) for downstream use.
Queuing / Scheduling	Deciding which queue's packets get transmitted next when a link is congested.
LLQ	Low Latency Queuing — a strict-priority queue for voice layered on top of CBWFQ for other classes.
Policing	Enforcing a rate limit by dropping or re-marking traffic that exceeds it immediately.
Shaping	Enforcing a rate limit by buffering and smoothly releasing excess traffic instead of dropping it.
RED / WRED	(Weighted) Random Early Detection — drops packets probabilistically before a queue fully fills, to avoid TCP global synchronization.
Trust Boundary	The network point where existing traffic markings are trusted, versus re-classified from scratch.
Jitter	Variation in packet delay — especially disruptive to real-time voice/video.

End of guide. QoS is fundamentally about making deliberate choices under scarcity — deciding in advance which traffic matters most when a link can't serve everything at once. The mental model worth keeping: classify and mark once, as close to the source as possible (Section 6), then let every device downstream simply trust and act on that decision.